

تقرير التكليف العملي: التنبؤ بدرجات الحرارة باستخدام

RNN

تطبيق عملي على بيانات السلاسل الزمنية (Time Series Data)

اسم الطالب: رغد اكرم بن يحيى
المادة: التعلم العميق / الذكاء الاصطناعي

1. مقدمة وهدف التكليف

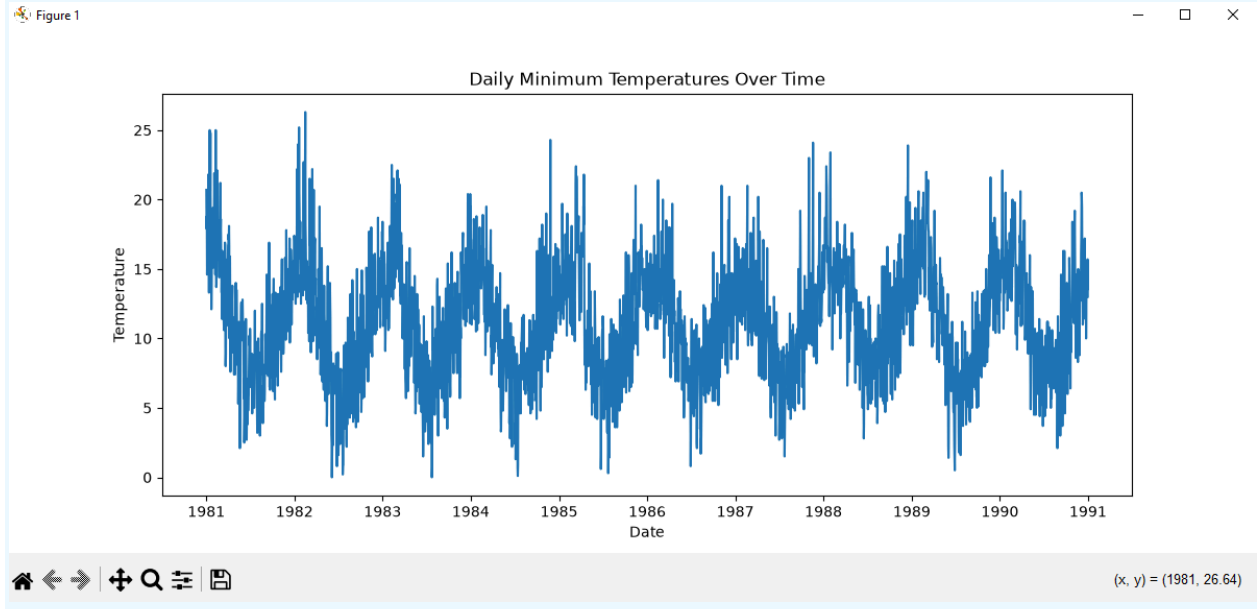
الهدف من التكليف هو بناء نموذج شبكة عصبية متكررة (SimpleRNN) للتنبؤ بدرجة الحرارة الدنيا لليوم التالي بالاعتماد على درجات الحرارة لـ 7 أيام سابقة، باستخدام مجموعة بيانات Daily Minimum Temperatures.

2. استكشاف وتجهيز البيانات (Data Exploration)

- إجمالي عدد السجلات: 3,650 سجل.
- القيم المفقودة: 0 (البيانات كاملة ولا توجد قيم مفقودة).
- تقسيم البيانات: تم التقسيم زمنياً (80% تدريب و 20% اختبار) دون خلط عشوائي للحفاظ على النمط الزمني.
- أبعاد المصفوفات النهائية:
 - بيانات التدريب: (X_train shape: (2914, 7, 1)
 - بيانات الاختبار: (X_test shape: (729, 7, 1)

3. عرض درجات الحرارة عبر الزمن

تم رسم البيانات لمعرفة سلوك وتغير درجات الحرارة عبر السنوات:



4. التطبيع (Normalization) والإجابة النظرية

تم استخدام MinMaxScaler لتحجيم درجات الحرارة بين 0 و 1.

سؤال: لماذا نستخدم Normalization قبل تدريب RNN؟

الإجابة:

1. تسريع تقارب النموذج وتقليل دالة الخسارة بشكل أسرع (Faster Convergence).
2. تفادي مشكلة تلاشي أو انفجار التدرجات (Vanishing/Exploding Gradients)، خصوصاً وأن دالة التنشيط $\tanh$ تعمل بأفضل كفاءة مع المدخلات المحصورة في نطاق ضيق ومحدد.

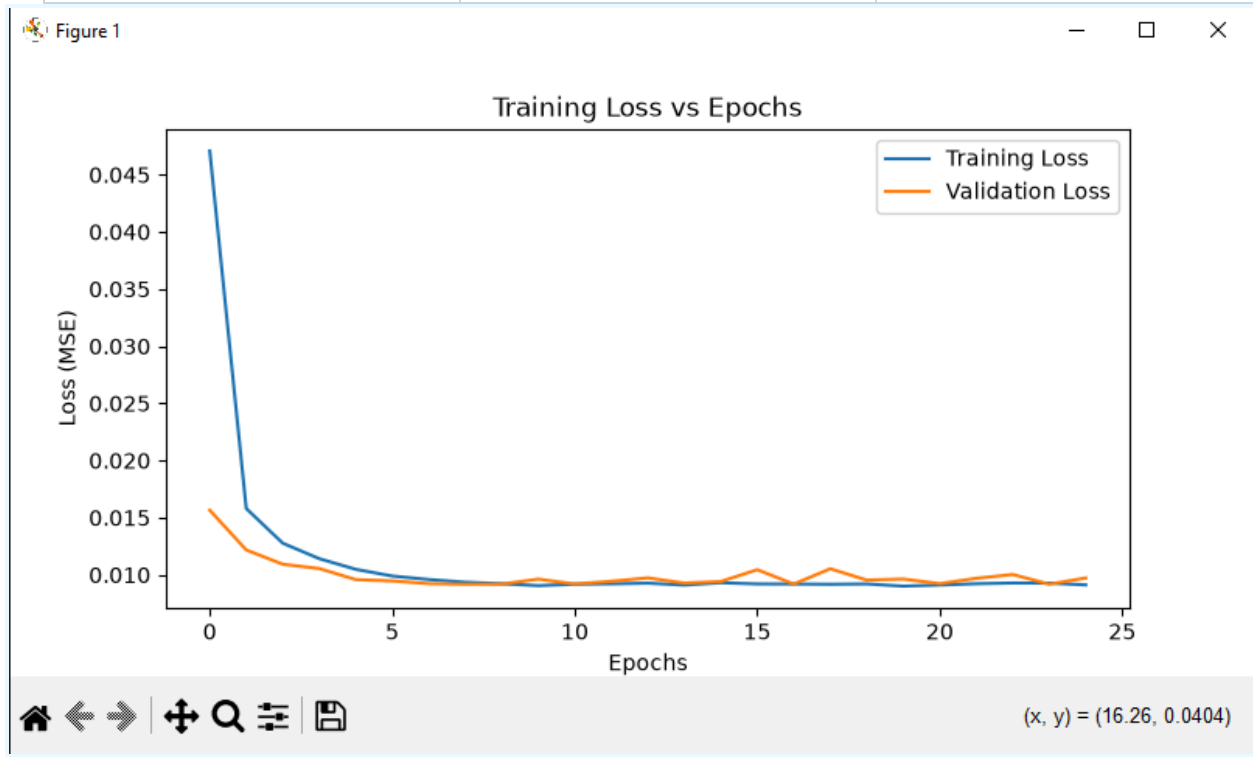
5. بنية النموذج والتدريب (Model Architecture & Training)

بنية النموذج المستخدم:

- (Input: (7, 1
- SimpleRNN: 32 وحدة مع تنشيط $\tanh$
- Dense: 1 وحدة للإخراج
- المحسن: Adam | دالة الخسارة: MSE | عدد الـ Epochs: 25

ملخص النموذج (Model Summary):

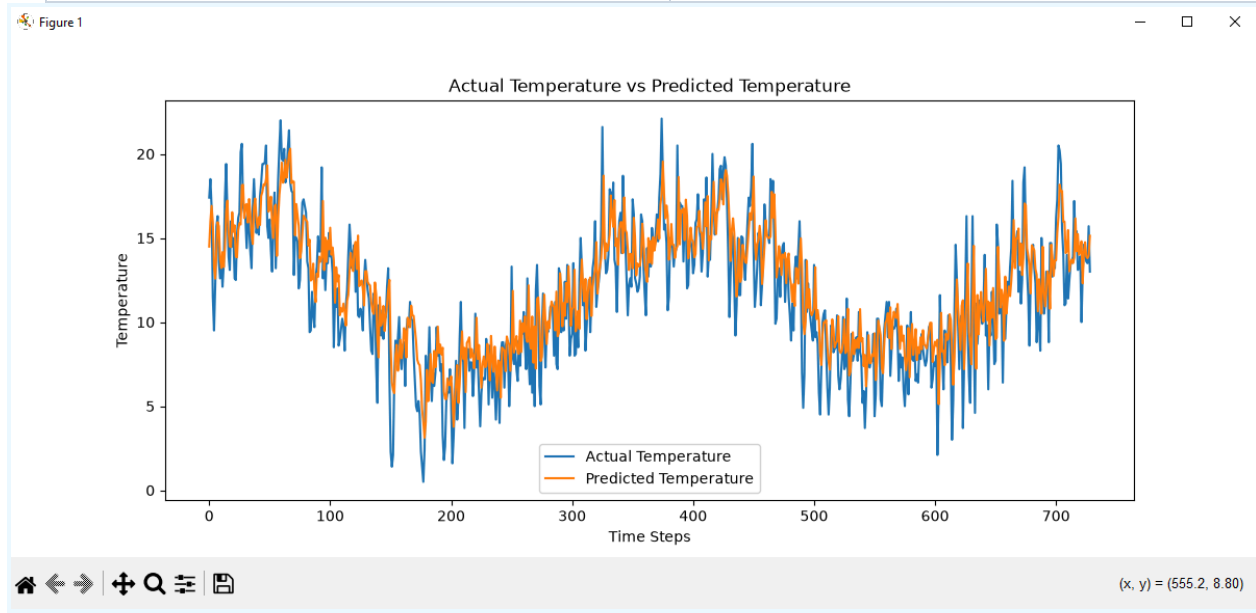
# Param	Output Shape	Layer
1,088	(None, 32)	(SimpleRNN (32 units
33	(None, 1)	(Dense (1 unit
(KB 4.38) 1,121		Total Params



6. تقييم أداء النموذج (Model Evaluation)

بعد إرجاع التوقعات إلى المقياس الطبيعي (Inverse Scaling)، كانت النتائج المحققة على بيانات الاختبار كالتالي:

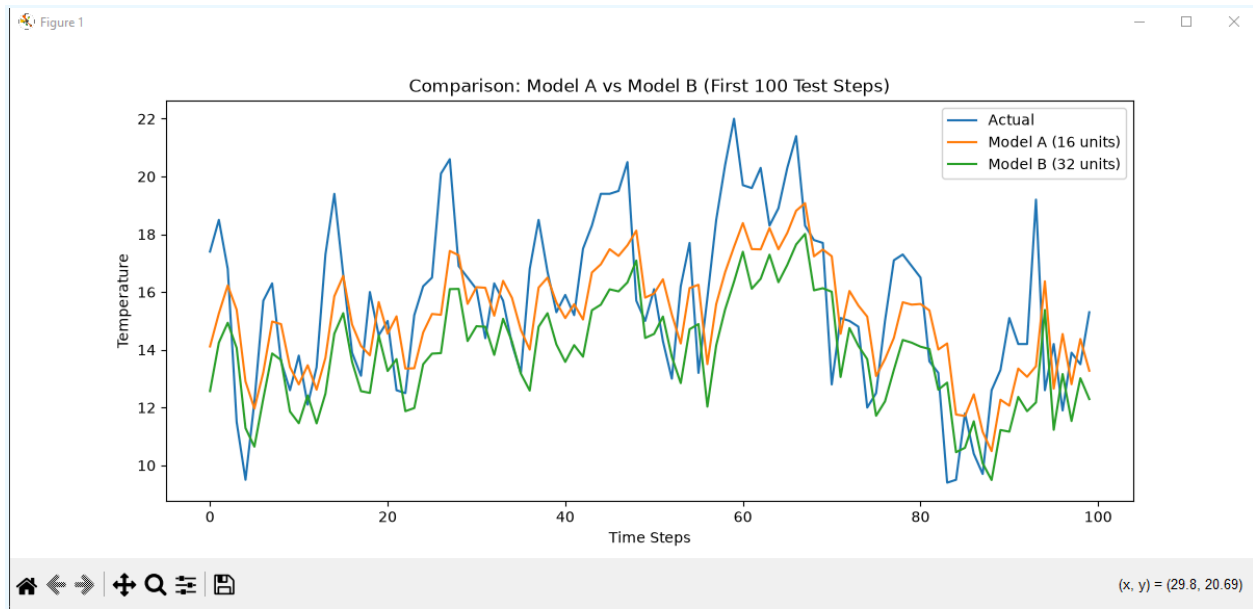
المقياس (Metric)	النتيجة
(Mean Absolute Error (MAE	1.797
(Mean Squared Error (MSE	5.168



7. التجربة الإضافية والمقارنة (Model A vs Model B)

مقارنة نموذج 16 وحدة مع نموذج 32 وحدة:

النموذج	الوحدات	Loss (التدريب)	MAE (الاختبار)	MSE (الاختبار)
Model A	Units 16	0.00908	1.764	5.004
Model B	Units 32	0.00907	1.838	5.356



الملاحظة والتحليل:

- نلاحظ أن زيادة الوحدات من 16 إلى 32 قللت الـ Training Loss بنسبة ضئيلة جداً، ولكن على بيانات الاختبار كان نموذج 16 وحدة أفضل بنسبة خطأ أقل (MAE: 1.764 مقابل 1.838).
- لماذا لا تعني زيادة الوحدات دائماً نموذجاً أفضل؟ زيادة عدد الوحدات ترفع تعقيد النموذج وقد تسبب Overfitting (حفظ بيانات التدريب بدلاً من التعميم)، مما يقلل من دقة التوقع على البيانات الجديدة، فضلاً عن استهلاك موارد حسابية ووقت أكبر دون فائدة حقيقية.

8. الكود البرمجي (Python Code)

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Input, SimpleRNN, Dense
```

```
df =  
pd.read_csv('https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv')
```

```
print(df.head())  
  
print(df.info())  
  
print(df.isnull().sum())  
  
print("Total records:", len(df))  
  
df['Date'] = pd.to_datetime(df['Date'])  
  
df['Temp'] = pd.to_numeric(df['Temp'], errors='coerce')  
  
df = df.dropna()  
  
plt.figure(figsize=(12, 5))  
  
plt.plot(df['Date'], df['Temp'])  
  
plt.title('Daily Minimum Temperatures Over Time')  
  
plt.xlabel('Date')  
  
plt.ylabel('Temperature')  
  
plt.show()  
  
data = df['Temp'].values.reshape(-1, 1)  
  
scaler = MinMaxScaler(feature_range=(0, 1))  
  
scaled_data = scaler.fit_transform(data)
```

```

def create_sequences(dataset, time_steps=7):

    X, y = [], []

    for i in range(len(dataset) - time_steps):

        X.append(dataset[i:(i + time_steps), 0])

        y.append(dataset[i + time_steps, 0])

    return np.array(X), np.array(y)

X, y = create_sequences(scaled_data, time_steps=7)

train_size = int(len(X) * 0.8)

X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)

model = Sequential([

    Input(shape=(7, 1)),

    SimpleRNN(32, activation='tanh'),

    Dense(1)

```

```
] )

model.summary()

model.compile(optimizer='adam', loss='mean_squared_error')

history = model.fit(X_train, y_train, epochs=25, batch_size=32,
                    validation_split=0.1)

plt.figure(figsize=(8, 4))

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Training Loss vs Epochs')

plt.xlabel('Epochs')

plt.ylabel('Loss (MSE)')

plt.legend()

plt.show()

predictions = model.predict(X_test)

predictions_rescaled = scaler.inverse_transform(predictions)

y_test_rescaled = scaler.inverse_transform(y_test.reshape(-1, 1))

mae = mean_absolute_error(y_test_rescaled, predictions_rescaled)

mse = mean_squared_error(y_test_rescaled, predictions_rescaled)

print("MAE:", mae)

print("MSE:", mse)
```



```
plt.figure(figsize=(12, 5))

plt.plot(y_test_rescaled, label='Actual Temperature')

plt.plot(predictions_rescaled, label='Predicted Temperature')

plt.title('Actual Temperature vs Predicted Temperature')

plt.xlabel('Time Steps')

plt.ylabel('Temperature')

plt.legend()

plt.show()


model_A = Sequential([

    Input(shape=(7, 1)),

    SimpleRNN(16, activation='tanh'),

    Dense(1)

])

model_A.compile(optimizer='adam', loss='mean_squared_error')

history_A = model_A.fit(X_train, y_train, epochs=25, batch_size=32,
verbose=0)


model_B = Sequential([

    Input(shape=(7, 1)),

    SimpleRNN(32, activation='tanh'),

    Dense(1)

])

model_B.compile(optimizer='adam', loss='mean_squared_error')
```

```
history_B = model_B.fit(X_train, y_train, epochs=25, batch_size=32,
verbose=0)

pred_A = scaler.inverse_transform(model_A.predict(X_test))
pred_B = scaler.inverse_transform(model_B.predict(X_test))

mae_A = mean_absolute_error(y_test_rescaled, pred_A)
mse_A = mean_squared_error(y_test_rescaled, pred_A)

mae_B = mean_absolute_error(y_test_rescaled, pred_B)
mse_B = mean_squared_error(y_test_rescaled, pred_B)

print(f"Model A (16 units) -> Loss:
{history_A.history['loss'][-1]:.5f}, MAE: {mae_A:.3f}, MSE:
{mse_A:.3f}")

print(f"Model B (32 units) -> Loss:
{history_B.history['loss'][-1]:.5f}, MAE: {mae_B:.3f}, MSE:
{mse_B:.3f}")

plt.figure(figsize=(12, 5))

plt.plot(y_test_rescaled[:100], label='Actual')

plt.plot(pred_A[:100], label='Model A (16 units)')

plt.plot(pred_B[:100], label='Model B (32 units)')

plt.title('Comparison: Model A vs Model B (First 100 Test Steps)')

plt.xlabel('Time Steps')

plt.ylabel('Temperature')
```

```
plt.legend()
```

```
plt.show()
```